

```
In [1]: # SIMPLE LINEAR REGRESSION

# 1.IMPORTING LIBRARY

import pandas as pd

# "import pandas" - refers importing all functions under pandas and it will be L
# pandas is a library function,it can handle all types of tables.It can read the
# It has multiple functions and also globalized for the users.
# as pd- means converting or assigning the word pandas as pd in simple.
# This will be saved once it run after execution.
```

```
In [2]: # DATA COLLECTION PHASE

dataset=pd.read_csv("Salary_Data.csv")

# pd=class
# read csv=function
# pd.read_csv refers - The function read_csv is called from the class pd.
# The dataset file is passed within the bracket with double quotes("Salary_Data.
# The file name sholud be same as mentioned in the database otherwise File not F
# The prticular dataset file should be under the same folder, here under SLR oth
```

```
In [3]: dataset

# To view datas or content in the dataset file, or can display using print state
# displays all the column and rows as input and output.
# In this salary dataset YearsExperience column is the input and Salary column i
# For this dataset preprocessing is not required becoz it is preprocessed data.
```

Out[3]:

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0
5	2.9	56642.0
6	3.0	60150.0
7	3.2	54445.0
8	3.2	64445.0
9	3.7	57189.0
10	3.9	63218.0
11	4.0	55794.0
12	4.0	56957.0
13	4.1	57081.0
14	4.5	61111.0
15	4.9	67938.0
16	5.1	66029.0
17	5.3	83088.0
18	5.9	81363.0
19	6.0	93940.0
20	6.8	91738.0
21	7.1	98273.0
22	7.9	101302.0
23	8.2	113812.0
24	8.7	109431.0
25	9.0	105582.0
26	9.5	116969.0
27	9.6	112635.0
28	10.3	122391.0
29	10.5	121872.0

In [4]:

#INPUT AND OUTPUT SPLIT UP PHASE

#INPUT SPLIT

```
independent=dataset[["YearsExperience"]]
```

```
# Assigning independent variable, also it is input.
```

```
# Input column ie., YearsExperience from the table is seperated or extracted or
```

```
# To fetch more than one column use comma follow by column name.
```

```
# Column name should be same as mentioned in the dataset without spelling mistake
```

In [5]: `independent`

```
# To print to check the content present in the independent variable or input column
```

Out[5]:

YearsExperience	
0	1.1
1	1.3
2	1.5
3	2.0
4	2.2
5	2.9
6	3.0
7	3.2
8	3.2
9	3.7
10	3.9
11	4.0
12	4.0
13	4.1
14	4.5
15	4.9
16	5.1
17	5.3
18	5.9
19	6.0
20	6.8
21	7.1
22	7.9
23	8.2
24	8.7
25	9.0
26	9.5
27	9.6
28	10.3
29	10.5

In [6]: *# OUTPUT SPLIT*

```
dependent=dataset[["Salary"]]
```

```
# Similarly the dependent refers output column, it is also seperately fetched fr  
# The column name should be same as such in dataset
```

In [7]: dependent

```
# To view the content or datas available in the output column  
# Both the i/p and o/p datas are seperated from the table
```

Out[7]:

	Salary
0	39343.0
1	46205.0
2	37731.0
3	43525.0
4	39891.0
5	56642.0
6	60150.0
7	54445.0
8	64445.0
9	57189.0
10	63218.0
11	55794.0
12	56957.0
13	57081.0
14	61111.0
15	67938.0
16	66029.0
17	83088.0
18	81363.0
19	93940.0
20	91738.0
21	98273.0
22	101302.0
23	113812.0
24	109431.0
25	105582.0
26	116969.0
27	112635.0
28	122391.0
29	121872.0

In [8]: *# TRAINING AND TEST SET SPLIT UP*

```
from sklearn.model_selection import train_test_split
```

```

X_train, X_test, Y_train, Y_test= train_test_split(independent,dependent,test_si

# sklearn is a library function.
# sklearn supports Machine Learning.
# it is importing only train and test function from the folder modelselection un
# Only required functions are called rather all functions like pandas, therefore

# X_train=training set input
# X_test =test set input
# y_train= training set output
# y_test = test set output

# the size of training and test set will be in the ratio 60:40,70:30,80:20
# underscore is used in middle of the word is to join and to make single word al
# For the function train_test_split the parameters are passed inside the bracket

# X means input and Y means output
# 4 variables created with the position, 1st- position is meant for X_train inp
#                               2nd-                               X_test inp
#                               3rd-                               y_train out
#                               4th-                               y_test outp

```

```

In [10]: # LEARNING PHASE

# ML algorithms are provided by sklearn library
# LinearRegression is called or imported from the function linear_model
# Spelling should be checked to avoid errors

# regressor=LinearRegression() means the function under LinearRegression is assi
#regressor.fit() means calling thr fit function through access operator
# fit(X_train,Y_train) means the value should given as input or the values to su
# SLR-  $y=wx+b$  where, weight & input + bais
# model is created by fit method
# LinearRegression()
# created model part is executed ie model is calculated

from sklearn.linear_model import LinearRegression
regressor=LinearRegression()
regressor.fit(X_train,Y_train)

```

```

Out[10]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)

```

```

In [12]: weight=regressor.coef_
weight

# The model value is checked by variable "weight=regressor.coef_" means the coef

```

```

Out[12]: array([[9360.26128619]])

```

```

In [13]: bais=regressor.intercept_

#To know the bais value
bais

```

```

Out[13]: array([26777.3913412])

```

```

In [15]: # EVALUATION

# The model,weight,bais values are saved in the variable regressor

```

```
# To evaluate the model test dataset is used ie., X_test  
# creating Y_pred variable and evaluation made for testdata ie., x_test  
  
Y_pred=regressor.predict(X_test)
```

```
In [16]: # function for finding r score value  
# r_score variable is created and for same function input is given as actual(y_test)  
  
from sklearn.metrics import r2_score  
r_score=r2_score(Y_test,Y_pred)  
  
# After running r score the result will be between 0-1, the model performance is
```

```
In [17]: r_score
```

```
Out[17]: 0.9740993407213511
```

```
In [18]: # TO SAVE MODEL  
  
#pickle is the funtion to save the model  
# assign filename as finalized_model_linear.sav  
# .sav is the extention filename  
# wb means write binary, rd means read binary  
  
import pickle  
filename="finalized_model_linear.sav"
```

```
In [19]: pickle.dump(regressor,open(filename,'wb'))
```

```
In [21]: # TO LOAD THE MODEL FOR CHECKING  
  
loaded_model=pickle.load(open("finalized_model_linear.sav",'rb'))  
result=loaded_model.predict([[13]])  
  
# loaded_model is the variable name created  
# pickle.load is the function to load. rb means read
```

```
In [22]: # To see the result from the variable name loaded_model  
result
```

```
Out[22]: array([[148460.78806172]])
```

```
In [ ]:
```